



CCM310

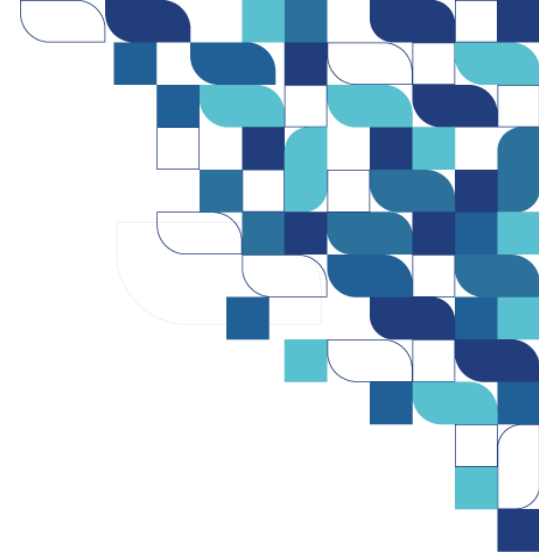
Arquitetura de Software e Programação Orientada a Objetos

Profa. Dra. Gabriela Oliveira Biondi

Departamento de Ciência da Computação FEI



JESUÍTAS BRASIL



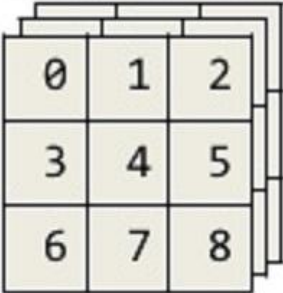
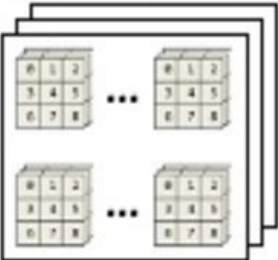
Array, ArrayList e Documentação

Aula 04

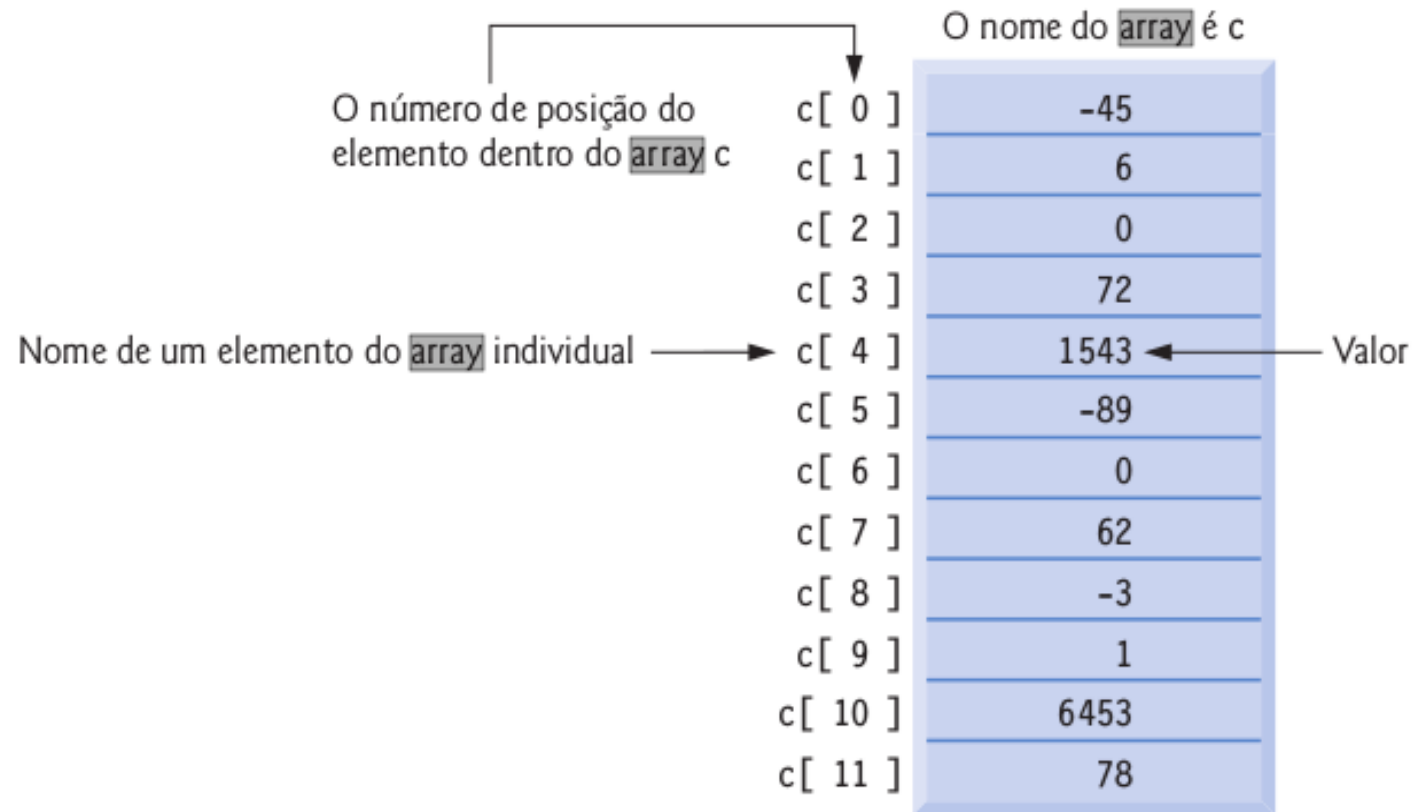
Array

- Coleções de itens de dados relacionados
- Consiste em dados do mesmo tipo
- Arrays são entidades de largura **fixa** (entidades estáticas)

What is an array?

Dimensions	Example	Terminology
1		Vector
2		Matrix
3		3D Array (3 rd order Tensor)
N		ND Array

Array – Exemplo de 12 elementos



Array – Declaração e Criação

```
int n[ ] = new int[12];  
// n é um array de 12 inteiros
```

```
int n[ ] = { 1, 2, 3, 4, 5 };  
// tamanho omitido
```

```
Pessoa[ ] n = new Pessoa[5];  
// array de objetos
```

Arrays Multidimensionais

```
int m[ ][ ] = { { 1, 2 }, { 3, 4 } };
```

```
int m[ ][ ];
```

```
m = new int [ 3 ] [ 4 ];
```

```
n[i] = new Pessoa();
```

Arrays Multidimensionais

	Coluna 0	Coluna 1	Coluna 2	Coluna 3
Linha 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Linha 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Linha 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Diagram illustrating the structure of a 2D array (matrix) with 3 rows and 4 columns. The array is represented as a grid of cells, each containing a specific element accessed via its row and column indices.

The elements are labeled as follows:

- Subscrito de coluna (Column subscript): Points to the second index (the value inside the second set of brackets).
- Subscrito de linha (Row subscript): Points to the first index (the value inside the first set of brackets).
- Nome do array (Array name): Points to the variable name 'a' at the beginning of the first element.

Exercício 1

- Crie um vetor (com 10 posições) para armazenar sensores a Laser (classe Laser):
- Crie a classe Laser. Cada Laser deve conter os seguintes atributos: fabricante, alcance, precisão e medida
- Depois de criados os 10 objetos Laser, leia as medidas obtidas por cada um deles

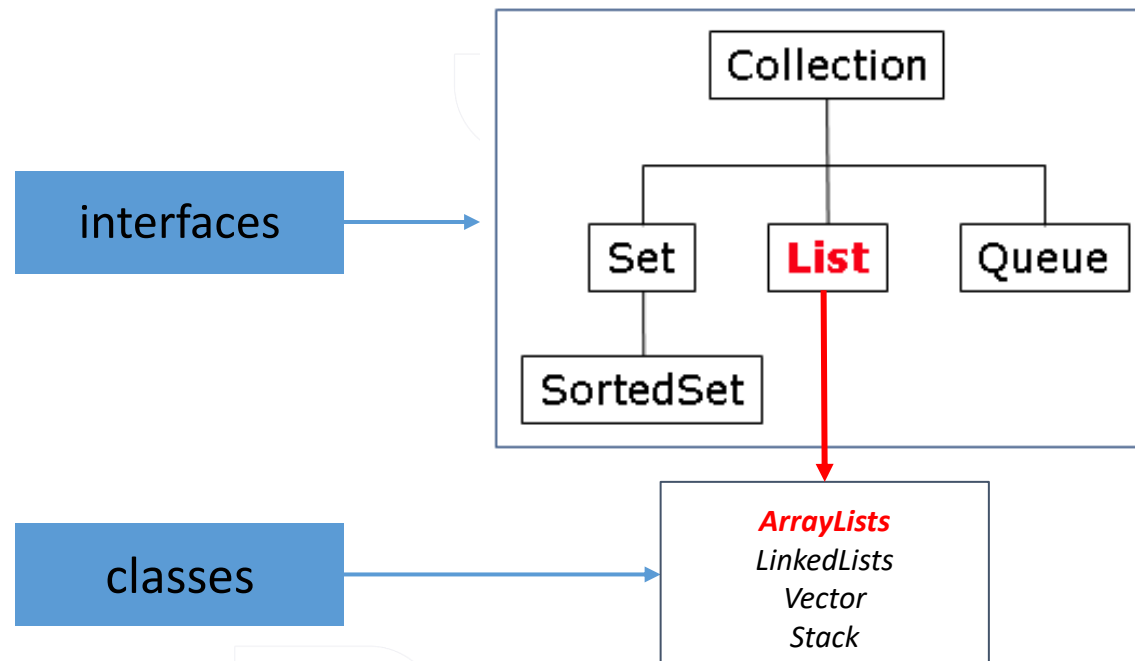
```
//vetor de objetos:  
Laser l[] = new Laser[10];
```

ArrayList

- ArrayLists são parte da estrutura de **coleções** do Java (Java Collections)
- Uma coleção é um objeto que implementa uma **estrutura de dados**, podendo armazenar referências a outros objetos
- Não armazena tipos primitivos diretamente, porém podemos utilizar as classes empacotadoras de tipo (Boolean, Integer, Float etc)

Coleções

- As classes e interfaces da estrutura de coleções são membros do pacote **java.util**



Coleções

- O **ArrayList** é, portanto, uma **classe**
- Precisamos instanciar um objeto ArrayList para utilizarmos seus métodos, que já estão implementados
- O ArrayList permite a manipulação dinâmica de dados!

Métodos do ArrayList (alguns)

Método	Descrição
add	Adiciona um elemento no fim de ArrayList
clear	Remove todos os elementos de ArrayList.
contains	Retorna true se ArrayList contiver o elemento especificado; do contrário, retorna false
get	retorna o elemento no índice especificado.
indexOf	Retorna o índice da primeira ocorrência do elemento especificado em ArrayList
remove	Remove a primeira ocorrência do valor especificado.
remove	Remove o elemento no índice especificado.
size	Retorna o número de elementos armazenados no ArrayList.
trimToSize	Reduz a capacidade de ArrayList de acordo com o número de elementos atual.

Exemplo

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3
4 class Pessoa{
5     public Pessoa(String nome, int idade)
6     {
7         this.nome = nome;
8         this.idade = idade;
9     }
10    public String getNome(){
11        return nome;
12    }
13    public int getIdade(){
14        return idade;
15    }
16
17    private String nome;
18    private int idade;
19};
```

```
class TestArrayList
{
    public static void main(String args[])
    {
        ArrayList <Pessoa> p = new ArrayList<>();

        p.add(new Pessoa("Fulano", 43));
        p.add(new Pessoa("Sicrano", 28));
        p.add(new Pessoa("Beltrano", 61));

        for (int i = 0; i < p.size(); i++){
            System.out.println("Nome: " + p.get(i).getNome());
            System.out.println("Idade: " + p.get(i).getIdade());
            System.out.println();
        }
    }
}
```

```
Nome: Fulano
Idade: 43

Nome: Sicrano
Idade: 28

Nome: Beltrano
Idade: 61
```

***for-each* loop**

- O Java tem o loop **for-each**, que é usado exclusivamente para percorrer os elementos em um **array**

- Sintaxe:

```
for (type variable : arrayname) {  
    // code block to be executed  
}
```

- Exemplo:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String i : cars) {  
    System.out.println(i);  
}
```

Exemplo *for-each*

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3
4 class Pessoa{
5     public Pessoa(String nome, int idade)
6     {
7         this.nome = nome;
8         this.idade = idade;
9     }
10    public String getNome(){
11        return nome;
12    }
13    public int getIdade(){
14        return idade;
15    }
16
17    private String nome;
18    private int idade;
19};
```

```
class TestArrayList
{
    public static void main(String args[])
    {
        ArrayList<Pessoa> p = new ArrayList<>();

        p.add(new Pessoa("Fulano", 43));
        p.add(new Pessoa("Sicrano", 28));
        p.add(new Pessoa("Beltrano", 61));

        for (Pessoa pessoa : p){
            System.out.println(pessoa.getNome());
            System.out.println(pessoa.getIdade());
            System.out.println();
        }
    }
}
```

```
Nome: Fulano
Idade: 43

Nome: Sicrano
Idade: 28

Nome: Beltrano
Idade: 61
```

Iteradores (*iterators*)

- As coleções em Java possuem também **iteradores**
- Iteradores são **objetos** que permitem ao programador **percorrer uma coleção**
- São usados para apontar para os elementos das coleções

Iteradores (*iterators*)

- Pacote **java.util.Iterator**

Método	Descrição
hasNext	Returns true if the iteration has more elements.
next	Returns the next element in the iteration.
remove	Removes from the underlying collection the last element returned by this iterator
forEachRemaining	Performs the given action for each remaining element until all elements have been processed or the action throws an exception

Exemplo com *iterator*

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3 import java.util.Iterator;
4
5 class Pessoa{
6     public Pessoa(String nome, int idade)
7     {
8         this.nome = nome;
9         this.idade = idade;
10    }
11    public String getNome(){
12        return nome;
13    }
14    public int getIdade(){
15        return idade;
16    }
17
18    private String nome;
19    private int idade;
20};
```

```
class TestArrayList
{
    public static void main(String args[])
    {
        ArrayList<Pessoa> p = new ArrayList<>();

        p.add(new Pessoa("Fulano", 43));
        p.add(new Pessoa("Sicrano", 28));
        p.add(new Pessoa("Beltrano", 61));

        Iterator<Pessoa> itr = p.iterator();

        while (itr.hasNext()){
            Pessoa pessoa = itr.next();
            System.out.println(pessoa.getNome());
            System.out.println(pessoa.getIdade());
            System.out.println();
        }
    }
}
```

```
Nome: Fulano
Idade: 43
```

```
Nome: Sicrano
Idade: 28
```

```
Nome: Beltrano
Idade: 61
```

Exercício 2

- Escreva um programa em Java que compara dois ArrayLists. Para isso, crie dois ArrayLists de String e, em cada um, insira nomes de cores.
- Então, compare ambos para ver se o conteúdo é igual. A sequência das cores não precisa ser a mesma.

```
import java.util.ArrayList;
```

```
ArrayList<String> lista = new ArrayList<>();
```

Exercício 3

- Escreva um programa em Java que copia um ArrayList para outro ArrayList. Para isso, crie um ArrayList de String e insira nomes. Então copie o ArrayList criado em outro ArrayList.

Documentação

- É um texto escrito que acompanha o software e geralmente explica como utilizá-lo
- Os textos podem também trazer diagramas que explicam o funcionamento e o relacionamento existente entre as funções, as classes ou partes específicas do código

Convenções e Boas Práticas de Programação

- 80% do custo para manter um software durante sua vida vai para manutenção
- Normalmente, a manutenção de softwares não é feita pelo autor original
- Convenções são importantes pois melhoram a legibilidade do software
- Se você for usar seu código-fonte como um produto, você precisa ter certeza de que ele é tão bem embalado e limpo como qualquer outro produto

Convenções e Boas Práticas de Programação

- Todas as linguagens de programação possuem um conjunto de convenções e boas práticas que devem ser seguidas para que o seu código tenha uma aparência profissional
- Aprenda os padrões das linguagens que você utilizar profissionalmente

Indentação

- Indentação (recuo) é um neologismo derivado da palavra em inglês ***indentation***
- Indentação é um termo aplicado ao código fonte de um programa para ressaltar ou definir a estrutura do algoritmo
- O código com indentação é mais legível

Indentação

Código em C com indentação:

```
if (unlikely(prev->policy == SCHED_RR))
    if (!prev->counter)
    {
        prev->counter = NICE_TO_TICKS(prev->nice);
        move_last_runqueue(prev);
    }
switch (prev->state)
{
    case TASK_INTERRUPTIBLE:
        if (signal_pending(prev))
        {
            prev->state = TASK_RUNNING;
            break;
        }
    default:
        del_from_runqueue(prev);
}
prev->need_resched = 0;
```

O mesmo código, **sem indentação**:

```
if (unlikely(prev->policy == SCHED_RR))
if (!prev->counter)
{
    prev->counter = NICE_TO_TICKS(prev->nice);
    move_last_runqueue(prev);
}
switch (prev->state)
{
    case TASK_INTERRUPTIBLE:
    if (signal_pending(prev))
    {
        prev->state = TASK_RUNNING;
        break;
    }
    default:
        del_from_runqueue(prev);
}
prev->need_resched = 0;
```

Indentação

- 4 espaços devem ser utilizados
- Evite linhas com mais de 80 caracteres
- Quando uma expressão não couber em uma única linha, quebre utilizando os seguintes princípios:
 - Quebre depois de uma vírgula
 - Quebre antes de um operador
 - Alinhe a nova linha com o começo da exp. de mesmo nível da linha anterior

```
function(longExpression1, longExpression2, longExpression3,  
        longExpression4, longExpression5);  
  
var = function1(longExpression1,  
                function2(longExpression2,  
                          longExpression3));
```

Documentação – Comentários Iniciais

- Todo código-fonte deve ser iniciado com um comentário que liste: o nome do programador, a data, a licença, uma breve descrição do propósito do programa e mais alguma informação que o programador achar relevante, ex: versão, projeto etc.
- No Java o comentário inicial segue o mesmo estilo adotado no C, exemplo:

```
/*  
 * Classname  
 *  
 * Version info  
 *  
 * Copyright notice  
 */
```

Documentação – Comentários Gerais

- Servem para explicar os programas para outros programadores (ou para você mesmo)
- São ignorados pelo compilador / interpretador
- No Java segue o seguinte padrão:
 - Comentário em Bloco – mesmo padrão do comentário inicial
 - Comentário de Linha Única: `/* Comment */`
 - Comentário de fim de linha: `// Comment`

Documentação – javadoc

- Cria a documentação dos códigos Java em HTML
- No terminal:
 - **javadoc *.java**
- Nas IDEs:
 - O método de chamar o javadoc varia, mas sempre é possível e, normalmente, simples

Documentação – javadoc

- Algumas tags muito úteis

@author	This tag lets you put the name of the code author into the documentation.
@parameter	This tag is used to define parameters that are passed into a method.
@return	This tag defines values that are returned from methods.
@see	This tag creates "See Also:" output. You'll normally use this tag to refer to related classes.
@version	This tag lets you define the version of the Java code you're developing. For instance, the code displayed in Listing 1 is considered to version 1.0.

Exemplo de Documentação

All Classes

Animal

MainAnimalPoli

Passaro

Peixe

Sapo

PACKAGE

CLASS

TREE

DEPRECATED

INDEX

HELP

PREV CLASS

NEXT CLASS

FRAMES

NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

Class Animal

java.lang.Object
Animal

Direct Known Subclasses:

Passaro, Peixe, Sapo

public class Animal

extends java.lang.Object

Constructor Summary

Constructors

Constructor and Description

Animal()

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type

Method and Description

void

move()

Exemplo de Documentação PDF

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Shape	4
Circunferencia	14
Cilindro	20
Paralelogramo	6
TresDimensoes	18
Cilindro	20
Paralelepipedo	24
Triangulo	9

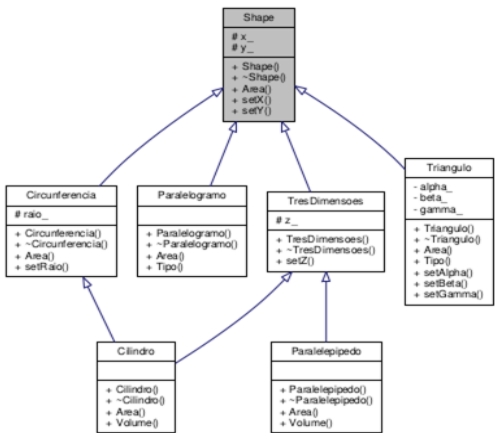
Chapter 4

Class Documentation

4.1 Shape Class Reference

```
#include <Shapes.h>
```

Inheritance diagram for Shape:



Shape
x_ # y_
+ Shape() + ~Shape() + Area() + setX() + setY()

Public Member Functions

- Shape ()
- ~Shape ()
- virtual float Area ()=0
- void setX (float x)
- void setY (float y)

Protected Attributes

- float x_
- float y_

4.1.1 Constructor & Destructor Documentation

4.1.1.1 Shape:Shape () [inline]

Constructor: initializes x_ with 2.0 and y_ with 2.0.

4.1.1.2 Shape::~Shape () [inline]

Destructor.

4.1.2 Member Function Documentation

4.1.2.1 virtual float Shape::Area () [pure virtual]

Implemented in Cilindro, Paralelepipedo, Circunferencia, Paralelogramo, and Triangulo.

4.1.2.2 void Shape::setX (float x)

Function member responsible for setting the x_ value.

Exemplo de Documentação HTML

My Project

Main Page	Classes	Files	Examples
Class List	Class Index	Class Hierarchy	Class Members

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

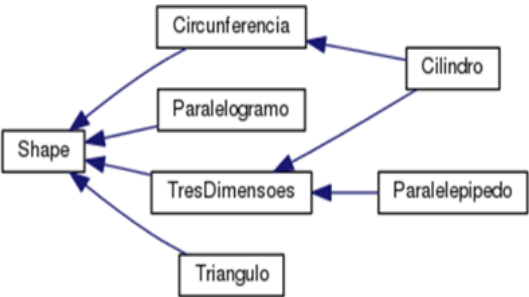
C Cilindro	Derived class Cilindro indicating inheritance relationship with class TresDimensoes and Circunferencia . Multiple inheritance
C Circunferencia	Derived class Circunferencia indicating inheritance relationship with class Shape
C Paralelepipedo	Derived class Paralelepipedo indicating inheritance relationship with class TresDimensoes
C Paralelogramo	Derived class Paralelogramo indicating inheritance relationship with class Shape
C Shape	Base-class
C TresDimensoes	Derived class TresDimensoes indicating inheritance relationship with class Shape
C Triangulo	Derived class Triangulo indicating inheritance relationship with class Shape

My Project

Main Page	Classes	Files	Examples
Class List	Class Index	Class Hierarchy	Class Members

Class Hierarchy

[Go to the textual class hierarchy](#)



Exercício 4

- Cria uma classe **ContaCorrente** com o atributo **saldo** e os métodos: **get** e **set**, **sacar** e **depositar**.
- Crie um **ArrayList** com 10 objetos da Classe **ContaCorrente** e utilize os métodos **sacar**, **depositar**, **setSaldo** e **getSaldo** de cada uma.
- Gere a documentação do exercício.

final

- Declara variáveis ~~constantes~~ de leitura
- Exemplos:

```
public final double pi = 3.1415926;
```

```
public final int x = 7;
```

```
public final class String{ } //classe não pode ser estendida (sem herança)
```

```
public final A ob = new A(); //Objeto não pode ser instanciado novamente
```

static

- São métodos que realizam tarefas que independem de objetos
- Estes métodos podem ser acessados pelo nome da classe, seguido do nome do método, sem nenhum objeto ter sido criado

```
1 class Teste{  
2     public static void metodo(){  
3         System.out.println("método chamado sem objeto ");  
4     }  
5 }  
6  
7 class Main{  
8     public static void main(String args[]){  
9         Teste.metodo();  
10    }  
11 }
```

static – variável de instância

- São atributos compartilhados entre todos os objetos da mesma classe
- Cada objeto não tem sua própria cópia deste atributo

```
1 class Teste{
2     public static int i = 0;
3     public void metodo(){
4         System.out.println("método chamado pelo objeto " + i++);
5     }
6 }
7
8 class Main{
9     public static void main(String args[]){
10         Teste t = new Teste();
11         t.metodo();
12         Teste t1 = new Teste();
13         t1.metodo();
14         Teste t2 = new Teste();
15         t2.metodo();
16     }
17 }
```

método chamado pelo objeto 0
método chamado pelo objeto 1
método chamado pelo objeto 2

Exercício 5

- Supondo que você vá programar um jogo de xadrez, construa uma classe **Peca**, com atributos **tipo**, **cor** e **posicao** (formada por coluna e linha - ex: "a7").
- Crie um **ArrayList** para armazenar objetos dinâmicos do tipo Peca.
- Armazene 32 peças, sendo 16 brancas e 16 pretas, com os seguintes tipos:
 - 8 peões ; 2 torres ; 2 cavalos ; 2 bispos ; 1 rainha e 1 rei
- Depois encontre uma peça pela posição e a remova do ArrayList.
- Imprima todos as peças restantes.

Exercício 6

- Construa uma classe **Estudante** com os seguintes atributos: **nome**, **sobrenome**, **identificador** e **proximoid**. O atributo proximoid é um número sequencial e é disponível para todos os estudantes.
- Faça os métodos **get** e **set** e um **construtor** parametrizado para a classe Estudante.
- Crie um **ArrayList** para representar uma turma com 30 alunos.
- Faça a impressão das informações dos estudantes.

Exercício 7

- Construa uma classe **Matematica**, que contém os seguintes métodos estáticos:
 - **max3()** – recebe três valores int como argumentos e retorna o valor do maior.
 - **impar()** – recebe três entradas booleanas e retorna verdadeiro se um número ímpar de entradas for true e falso caso contrário.
 - **maioria()** – recebe três argumentos booleanos e retorna verdadeiro se pelo menos dois dos argumentos tiverem o valor true e falso caso contrário. Não utilize if.
- Teste os métodos da classe no main()

Exercício 8

- Escreva uma classe **Pessoa** que tenha como variáveis de instância o **nome** (private) de uma pessoa, o seu **telefone** (private) e um **id** sequencial único.
- O **construtor** dessa classe deve ter como parâmetros o nome e o telefone. O **id** deve ser gerado automaticamente.
- Em seguida, escreva um programa que tenha como entrada as seguintes opções:

Exercício 8

- **n:** Cria uma nova pessoa na agenda:
 - solicita o nome: entre com o nome da pessoa que será criada
 - solicita o telefone: entre com o telefone da pessoa
- **d:** Deleta uma pessoa da agenda:
 - solicita o nome: entre com nome da pessoa que será excluída
- **p:** Imprime toda a agenda
- **q:** sai do programa

Exercício 8

```
Entre com uma das seguintes opções:  
  n [nova entrada]  
  d [apaga registro da agenda]  
  p [imprime toda a agenda]  
  q [sai do programa]  
  
n  
Entre com o nome:  
Fulano  
Entre com o telefone:  
12345678  
  
Entre com uma das seguintes opções:  
  n [nova entrada]  
  d [apaga registro da agenda]  
  p [imprime toda a agenda]  
  q [sai do programa]
```

Utilize um loop infinito até que *q* seja a entrada.

Armazene todos os objetos do tipo *Pessoa* em um *ArrayList* e manipule os objetos pelo *ArrayList*.

PS: considere que as pessoas nunca terão nomes iguais para facilitar a busca.

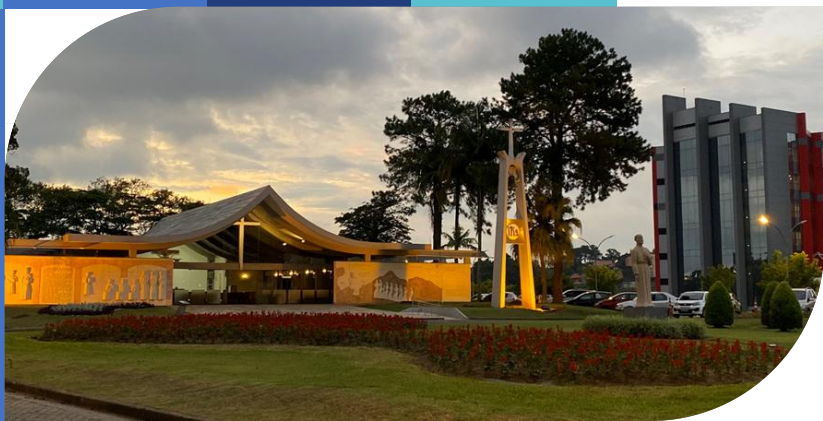
Exercício 9

- Construa uma classe **Carro**, composta por objetos de várias outras classes: **Roda**, **Motor**, **Porta**, **Banco**, **Volante**, etc.
- Crie pelo menos 10 classes para compor Carro. Cada classe deve ter **2 atributos** no mínimo, métodos **get** e **set** e **toString**.
- Muitos objetos que compõe carro serão manipulados por meio de **ArrayLists** (dentro da classe Carro), como objetos da classe Roda, por exemplo, já que a maioria dos carros é composto por 4 rodas.
- No main, crie um ArrayList de Carros e instancie 3 Carros.
- Faça a impressão das informações de todos os 3 Carros.



**Obrigada pela sua
participação, nos vemos na
próxima aula!**

:)



FEJ.
edu.br



JESUÍTAS BRASIL